

ISSTA 2026 · OAKLAND, CALIFORNIA

Evaluating and Mitigating the Misguidance Effect of Buggy Code in LLM-Generated Unit Tests

Junda Zhao¹ · Shurui Zhou² · Eldan Cohen¹

¹ Mechanical & Industrial Engineering ² Electrical & Computer Engineering
University of Toronto



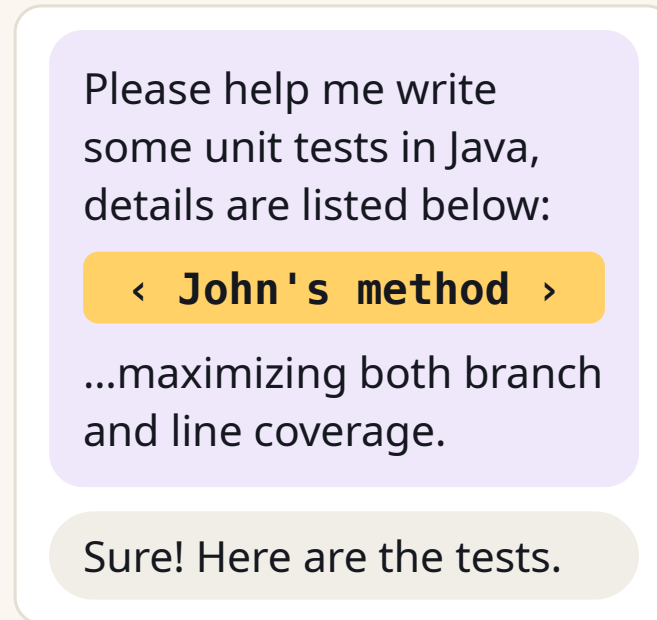
AN ORDINARY DAY

Just another day for John.

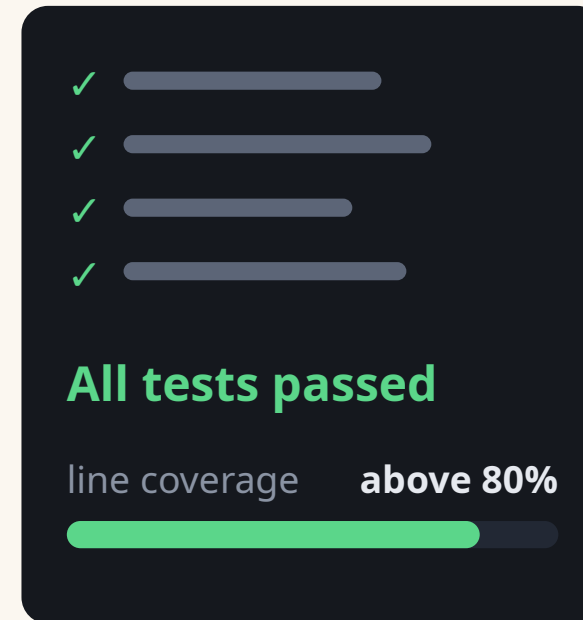
1 finishes his method



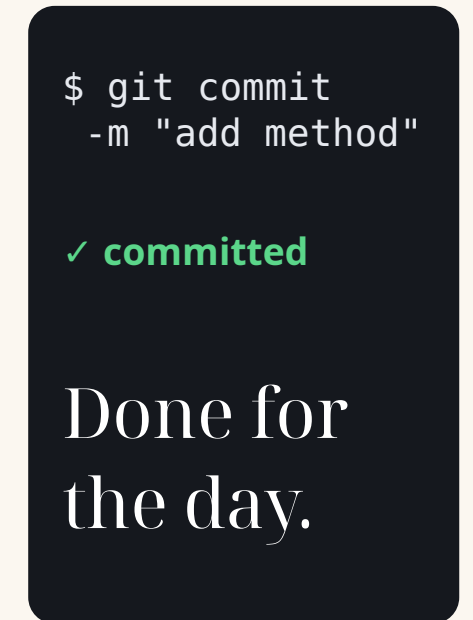
2 asks the LLM for tests



3 sees a green suite



4 commits



Anything feel wrong?

A **mine** has just been buried.

The tests are green because they agree with the code,
and the code has a bug.



THE MINE

A test that learned the bug.

John's method: `StringUtils.equals` (Lang-14)

```
public static boolean equals(  
    CharSequence cs1, CharSequence cs2) {  
    if (cs1 == cs2)          return true;  
    if (cs1 == null  
        || cs2 == null)      return false;  
    return cs1.equals(cs2);  
}
```

It calls `cs1.equals(cs2)`,
and `String.equals` is false for any non-String.

The test the LLM generated from it

```
assertFalse(StringUtils.equals("abc",  
    new StringBuilder("abc")));  
  
// the buggy body returns false here,  
// since cs2 is a non-String object,  
// and the LLM took that as intended.
```

✓ on John's buggy code
✗ on the bug-free code

A misguided test: it passes on the buggy code
and fails on the bug-free code.

AND THIS IS NOT JUST JOHN

Most prior studies do exactly the same thing.

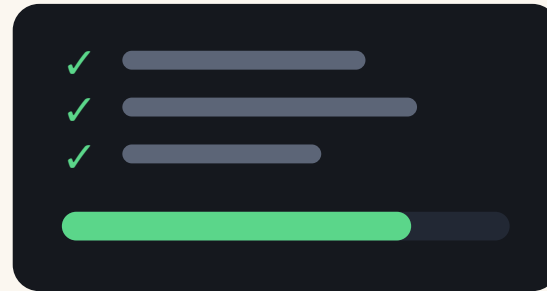
GOES IN

The code under test,
and it is **bug-free** code.



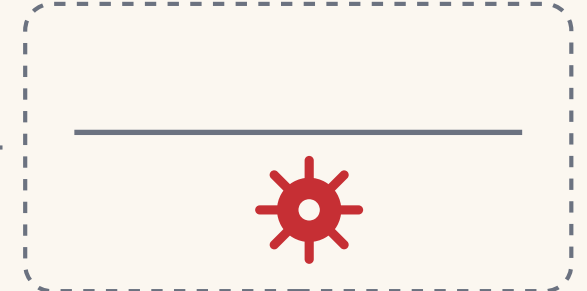
COMES OUT

Test correctness
and coverage.



RARELY MEASURED

Whether the tests
detect real bugs.



Prior work observed that buggy code weakens generated tests,
but **none has quantified it accurately**.

No effective mitigation has been proposed for such an effect
when ground-truth intent is missing.

How bad is it?

You need both versions to know.

Are all tests that pass on buggy code misguided?

A prior study (Huang et al.) counts them all, but on average 93.77% are correct tests that also pass on bug-free code.

An inaccurate measure of misguidance.

We also count the effective tests, to see how misguidance affects bug detection.

		Outcome on buggy code	
		Pass	Fail
Outcome on bug-free code	Pass	True negative A valid test the bug does not affect	Effective Detects the bug
	Fail	Misguided Asserts the bug	False positive Asserts behavior in neither version

SETUP

How we designed our study.

BENCHMARK

318

focal methods

from 233 bugs of Defects4J v3.0, each in a buggy and a fixed version

Every generated test is compiled and run on both versions

MODELS

11

LLMs

13 configurations:
6 base, 7 reasoning
Prompt adapted from Yang et al.

RESEARCH QUESTIONS

- RQ1** How does the misguidance effect of buggy code affect the behavior of LLM-generated unit tests?
- RQ2** How does our proposed specification-based unit test generation pipeline help mitigate the misguidance effect?
- RQ3** Does our proposed specification-based unit test generation pipeline impact unit tests generated from bug-free code?

BASE

Gemini 2.5 Flash · Claude 4 Sonnet · Grok-3 · GPT-4.1 · DeepSeek-V3 · Qwen3-Coder-Plus

REASONING

Gemini 2.5 Pro · Gemini 2.5 Flash · Claude 4 Sonnet · Grok-4 · GPT-O4-mini · DeepSeek-R1 · Qwen3-Plus

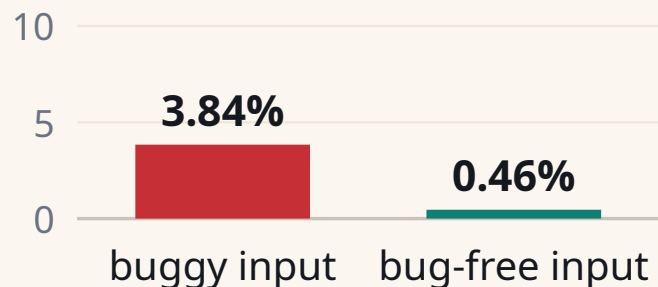
RQ1 • THE EFFECT

Buggy code misguides the LLM, and cuts real bug detection.

8× more misguided tests
from buggy code

Misguided tests

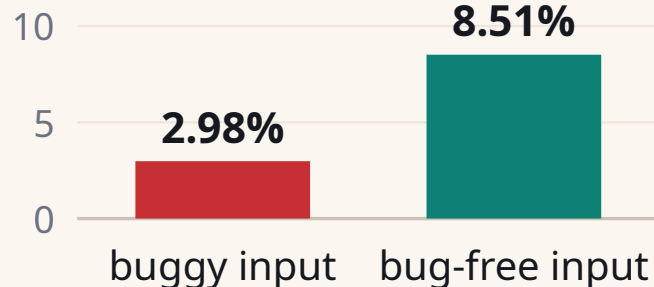
% of generated tests



65% fewer effective tests
from buggy code

Effective tests

% of generated tests



Capability and susceptibility go together

Models that generate more effective tests from bug-free code generate more misguided ones from buggy code ($r = 0.90$).

This suggests the most capable models can also be the most susceptible.

Evaluations on bug-free code **likely overestimate** real bug detection on buggy code.

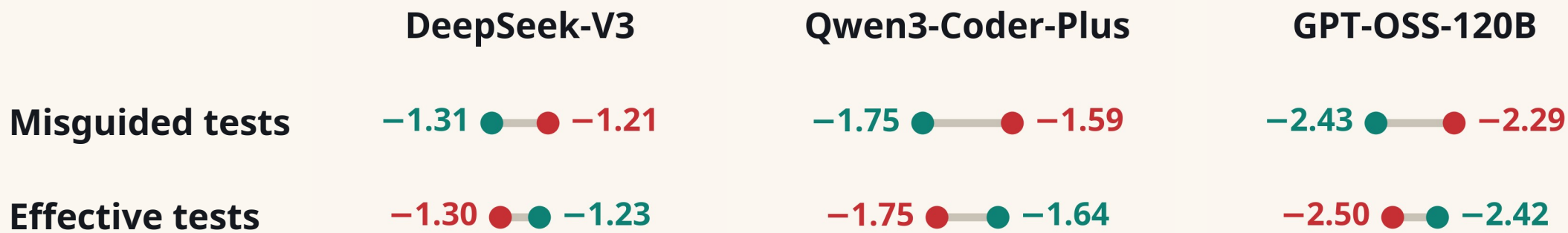
The preference shift is inside the model.

Sequence score: mean per-token log-probability of a test — the model's preference.

$$S(T, C) = \frac{1}{|T|} \sum_{i=1}^{|T|} \log P(t_i \mid C, t_1, \dots, t_{i-1})$$

● prompt holds buggy code

● prompt holds bug-free code



mean log-probability per token (higher = preferred)

Buggy code in the prompt **shifts the model's preference** toward the misguided tests.

Stop feeding it ~~the code~~.

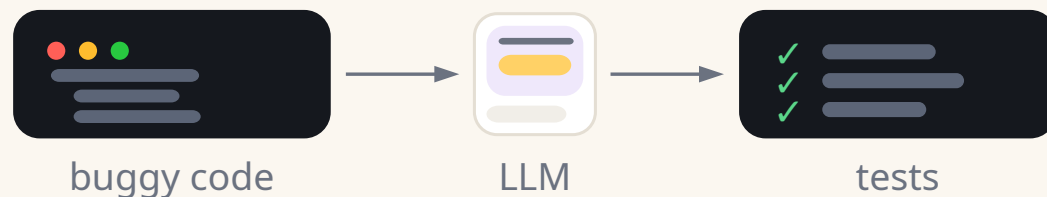
Feed it the intent.

Specification-based testing, as in TDD:
tests come from the intended behavior,
not from the code.

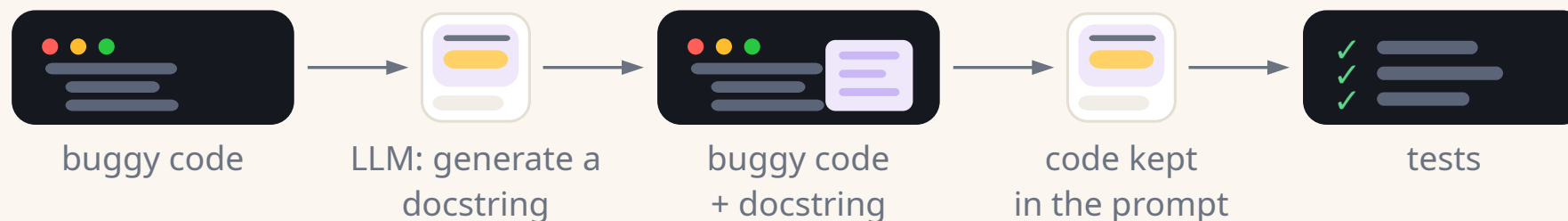
We let an LLM recover the specification.

Replace the code with its specification.

Today



Supplement
(Yuan et al.)



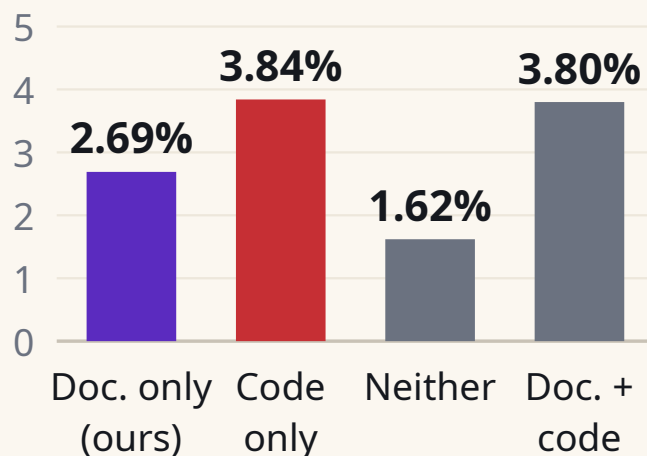
Ours



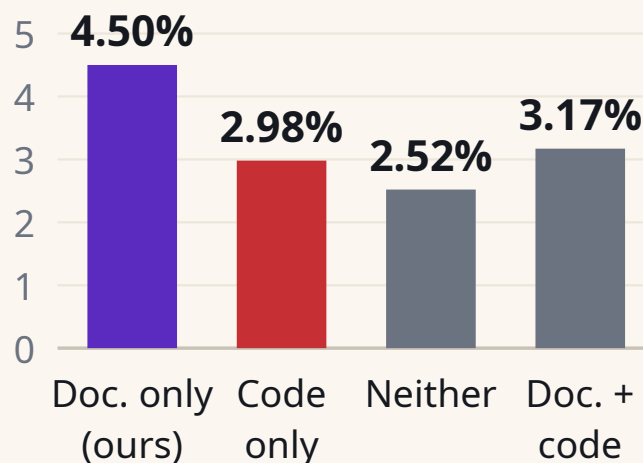
To mitigate misguidance, we **fully remove the potentially buggy code** and replace it with an LLM-generated specification.

Replacing works; supplementing does not.

Misguided tests (%)



Effective tests (%)



Ours vs code

73.31 vs 45.46 bugs detected, 47.62 vs 62.46 suites with misguided tests.

'Neither' only looks clean

Only 56.69% of its tests pass on either version (ours: 81.05%).

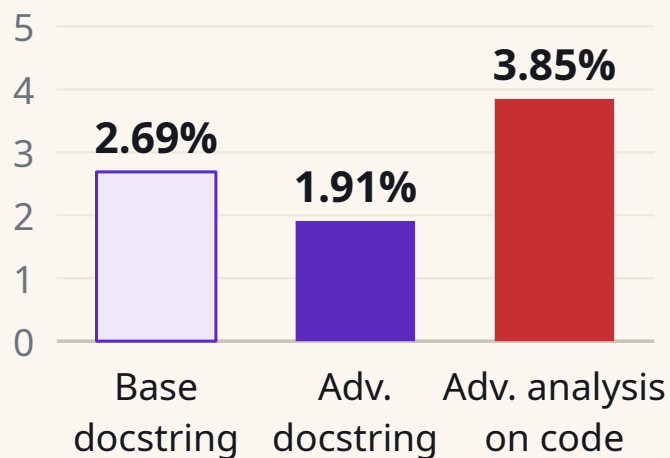
Docstring + code barely moves

+0.19 pp effective over code only, and no reduction in misguided tests.

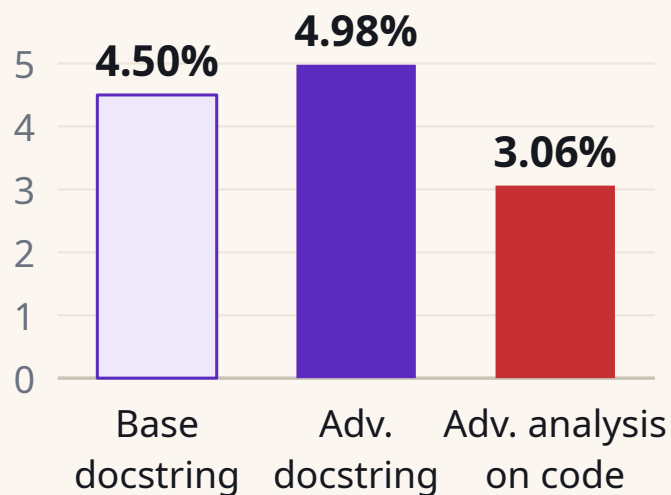
Replace the buggy code in the prompt with the **recovered specification**.

A better specification helps more.

Misguided tests (%)



Effective tests (%)



Advanced docstring prompt

Find logical errors and robustness gaps, propose a fix for each, then describe the version with all fixes applied.

Bugs detected vs base docstring

+18.88%

Misguided suites vs base docstring

-12.60%

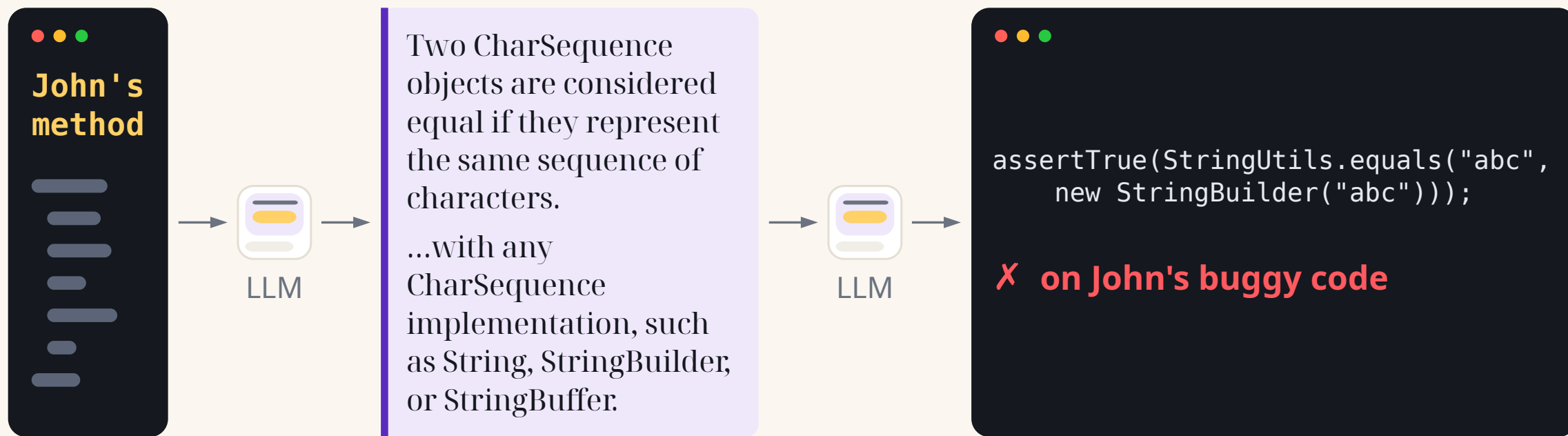
The analysis must be **combined with the specification** to pay off.

THE SAME METHOD

John's day, again.

1 his method 2 the LLM recovers the specification

3 the test generated from the specification alone

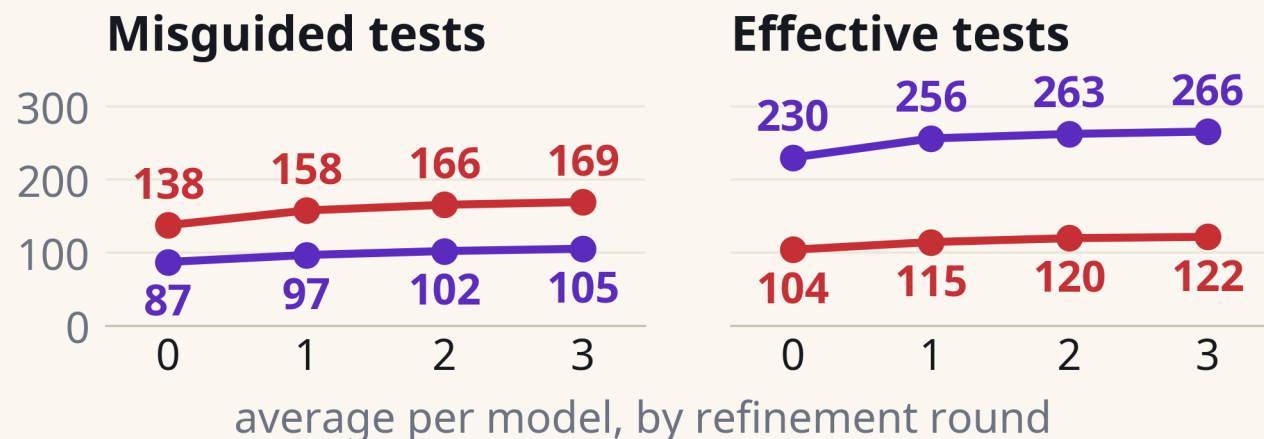


The buried mine is cleared **before the commit.**

The gain accumulates through refinement.

ChatTester-style loop, 3 rounds: a test that fails to execute is fed back with its error.

● code input ● docstring input (ours)



+73.11%

More bugs newly detected
9.85 vs 5.69 per model, ours vs code

-60.79%

Fewer suites with misguided tests
3.38 vs 8.62 per model, ours vs code

Refinement helps both inputs compile; the docstring adds **far fewer misguided tests.**

What the specification carries decides what the test finds.

636 advanced docstrings inspected (Gemini 2.5 Pro, Qwen3-Coder-Plus),
two annotators, κ 0.82 / 0.84 and 0.77 / 0.79

	GEMINI 2.5 PRO	QWEN3-CODER-PLUS
Preserve the bug	15.09%	20.75%
Recover the corrected behavior	62.26%	47.48%

Inherit fewer bugs → fewer misguided tests; recover more behavior → **find more bugs.**

And if the code
was **bug-free**?

Little to no penalty on bug-free code.

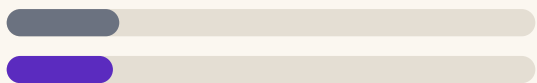
Same pipeline applied to the bug-free code; baseline = prompting with the bug-free code itself.

■ baseline: bug-free code in the prompt ■ ours: recovered specification

Compile-failure rate

−1.20 pp

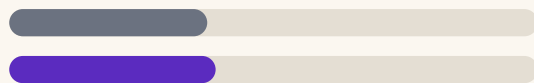
21.30% → 20.10%



False-alarm rate

+1.59 pp

37.45% → 39.04%



Line coverage

+3.61 pp

71.49% → 75.10%



Branch coverage

+2.97 pp

66.61% → 69.58%



Applicable to **both buggy and bug-free code**:
no extra burden, and suitable for tasks like regression testing.

Takeaways.

- 1 Buggy code misguides the LLM: more tests assert the bug, fewer catch it.
- 2 Feed the specification, not the code, even if LLM-generated from the code.
- 3 More accurate specifications further improve our approach.
- 4 Little to no penalty on bug-free code.
- 5 **Directly measure bug detection capability of tests from buggy code.**

Thank you

Feel free to contact me:

junda.zhao@mail.utoronto.ca

I am on the job market for postdoc positions.



Paper



Replication package



RQ1 per model: misguided and effective tests

Model	Buggy code					Bug-free code				
	Total	Misg. #	Misg. %	Eff. #	Eff. %	Total	Misg. #	Misg. %	Eff. #	Eff. %
Gemini 2.5 Flash	3539	114	3.22	140	3.96	3616	11	0.30	279	7.72
Claude 4 Sonnet	4202	149	3.55	88	2.09	4250	15	0.35	361	8.49
Grok-3	2543	71	2.79	93	3.66	2581	17	0.66	199	7.71
GPT-4.1	2841	92	3.24	86	3.03	2905	11	0.38	236	8.12
DeepSeek-V3	3002	76	2.53	113	3.76	3008	19	0.63	241	8.01
Qwen3-Coder-Plus	3455	80	2.32	157	4.54	3353	10	0.30	259	7.72
Base average	3263.67	97.00	2.94	112.83	3.51	3285.50	13.83	0.44	262.50	7.96
Gemini 2.5 Pro	3284	173	5.27	98	2.98	3286	17	0.52	354	10.77
Gemini 2.5 Flash (Reason)	3957	155	3.92	99	2.50	3956	7	0.18	293	7.41
Claude 4 Sonnet (Reason)	4515	187	4.14	88	1.95	4553	27	0.59	372	8.17
Grok-4	3923	179	4.56	145	3.70	3985	19	0.48	398	9.99
GPT-O4-mini	2500	130	5.20	36	1.44	2606	7	0.27	248	9.52
DeepSeek-R1	3413	145	4.25	97	2.84	3622	21	0.58	324	8.95
Qwen3-Plus	4857	239	4.92	114	2.35	4808	33	0.69	389	8.09
Reasoning average	3778.43	172.57	4.61	96.71	2.54	3830.86	18.71	0.47	339.71	8.99
Total average	3540.85	137.69	3.84	104.15	2.98	3579.15	16.46	0.46	304.08	8.51

Models evaluated

Developer	Model	Type	Open weights	Configs
Google	Gemini 2.5 Pro	Reasoning	No	1
	Gemini 2.5 Flash	Hybrid	No	2
Anthropic	Claude 4 Sonnet	Hybrid	No	2
xAI	Grok-4	Reasoning	No	1
	Grok-3	Base	No	1
OpenAI	GPT-4.1	Base	No	1
	GPT-O4-mini	Reasoning	No	1
DeepSeek	DeepSeek-V3	Base	Yes	1
	DeepSeek-R1	Reasoning	Yes	1
Alibaba	Qwen3-Coder-Plus	Base	Yes	1
	Qwen3-Plus	Reasoning	Yes	1

Hybrid models run in both modes, so 11 models give 13 configurations.

Prompt templates

●●● Test-generation prompt

You are a professional who writes Java test methods. Please help me write some unit tests in Java language, details are listed below:

<code under test and/or LLM-generated docstring, and other focal-method context>

Please write some unit tests in Java 11 and JUnit 4 with maximizing both branch and line coverage. Please ensure that the output format is Markdown, and no explanations needed.

Shared body: You are a professional Java developer. Please help identify the intention and functionality of the method detailed below: <code under test and related information>

Docstring prompt = the shared body + one postfix (base or advanced).

Advanced analysis: First perform a critical analysis as an expert auditing for quality and correctness, identifying (1) logical mistakes and (2) robustness omissions such as missing null checks, boundary conditions, error handling, sanitization or escaping; then describe the specification of a correct and robust version. Base models are additionally asked to list the fixes before writing the docstring.

Base postfix: Please provide a formal docstring that identifies the functionality and intention of the given method. Please avoid directly quoting from the focal method code, as it might be buggy, and output only the docstring.

Base vs advanced docstring on the same buggy method

● ● ● Bug-free code

```
public String generateToolTipFragment(
    String toolTipText) {
    return " title=\"\" +
        ImageMapUtilities.htmlEscape(toolTipText)
        + "\" alt=\"\"";
}
```

BASE DOCSTRING PROMPT OUTPUT (ABRIDGED)

Generates an HTML fragment for a tooltip attribute in an image map. The method constructs a standardized HTML title attribute along with an empty alt attribute. @param toolTipText The text content to be displayed as the tooltip. **This should be properly escaped for HTML inclusion.**

✗ 0 effective tests, 3 misguided tests

● ● ● Buggy code

```
public String generateToolTipFragment(
    String toolTipText) {
    return " title=\"\" +
        toolTipText
        + "\" alt=\"\"";
}
```

ADVANCED DOCSTRING PROMPT OUTPUT

Generates an HTML fragment for a tooltip attribute (`title`) and an empty `alt` attribute. **The method ensures the tooltip text is properly escaped** to prevent HTML syntax errors or security vulnerabilities, e.g., XSS. If the input is `null`, an empty `title` attribute is returned.

✓ 5 effective tests, 0 misguided tests

Base docstring: the caller escapes. Advanced: the method escapes, so only its tests catch the bug.

Method-level results

Model	Doc. only (ours)		Code only		Neither		Doc. + code	
	Misg.	Eff.	Misg.	Eff.	Misg.	Eff.	Misg.	Eff.
Gemini 2.5 Flash	47	74	44	51	21	50	49	59
Claude 4 Sonnet	52	85	59	41	26	58	59	59
Grok-3	32	60	46	43	26	59	44	43
GPT-4.1	42	73	56	45	24	52	61	48
DeepSeek-V3	39	65	42	51	28	57	40	60
Qwen3-Coder-Plus	42	73	43	58	26	42	44	55
Gemini 2.5 Pro	53	86	89	45	27	80	85	42
Gemini 2.5 Flash (Reason)	54	57	64	38	34	48	65	33
Claude 4 Sonnet (Reason)	49	83	63	42	33	64	63	58
Grok-4	53	93	84	60	34	70	76	65
GPT-O4-mini	49	61	72	29	22	53	69	41
DeepSeek-R1	54	68	69	45	36	42	59	53
Qwen3-Plus	53	75	81	43	41	46	75	58
Average	47.62	73.31	62.46	45.46	29.08	55.46	60.69	51.85

Model	Base docstring		Adv. docstring		Adv. test	
	Misg.	Eff.	Misg.	Eff.	Misg.	Eff.
Gemini 2.5 Flash	47	74	38	75	53	51
Claude 4 Sonnet	52	85	42	105	69	55
Grok-3	32	60	38	61	43	49
GPT-4.1	42	73	35	87	57	44
DeepSeek-V3	39	65	35	67	50	66
Qwen3-Coder-Plus	42	73	40	86	53	54
Base average	42.33	71.67	38.00	80.17	54.17	53.17
Gemini 2.5 Pro	53	86	43	100	90	50
Gemini 2.5 Flash (Reason)	54	57	44	78	65	32
Claude 4 Sonnet (Reason)	49	83	42	106	65	51
Grok-4	53	93	47	113	69	91
GPT-O4-mini	49	61	35	82	84	41
DeepSeek-R1	54	68	51	77	77	63
Qwen3-Plus	53	75	51	96	79	52
Reasoning average	52.14	74.71	44.71	93.14	75.57	54.29
Total average	47.62	73.31	41.62	87.15	65.69	53.77

A false alarm from a hallucinated docstring

● ● ● Bug-free focal method

```
public String generateToolTipFragment(
    String toolTipText) {
    return " title=\"\" +
        ImageMapUtilities.htmlEscape(toolTipText)
        + "\" alt=\"\"";
}
```

```
/**      Generated docstring with hallucinated behavior
 * Generates a sanitized HTML attribute fragment...
 * <li>Gracefully handle null... treating null as an empty
 * string rather than producing a literal "null" string
 * or throwing an exception.</li>
 * @return ... or an empty string if input is null
 */
```

● ● ● Test asserting the hallucinated behavior

```
@Test
public void testGenerateToolTipFragment_Null() {
    String result = gen.generateToolTipFragment(null);
    Assert.assertEquals("", result);
}
```

The cost of a better prompt

Hallucinated tests rise slightly with the advanced docstring (16.19% → 18.15%).

The docstring invented a null contract, the test asserts it, and the correct method fails it: a false alarm.

Threats to validity

External

Defects4J only (318 methods, 17 projects, 11 LLMs). Domain-specific or deeply stateful code may need more context than a method-level docstring carries.

Internal

Same prompts, parameters and pipeline for every model. Manual labels from two annotators with Cohen's κ and discussion to consensus.

Construct

Temperature 0 where possible. Contamination: the docstring acts as a paraphrase and improved every model. Hallucination measured as false-positive and false-alarm tests.

Threats: benchmark scope, one pipeline for every model, hallucination measured as false positives.